



Programa de Ingeniería de Sistemas Reto Hackathon – Campus Party FUP

Área responsable:	FUP Programa Ingeniería de Sistemas – Hospital Susana López de Valencia
Nombre de la Persona de contacto:	
Cargo, correo y teléfono del contacto:	

Caso de estudio: *El hospital Susana López de Valencia de mediana complejidad en proceso de modernización enfrenta serios desafíos en la gestión de su operación diaria. Atiende un promedio de 500 pacientes diarios en sus servicios de urgencias, hospitalización y consulta externa. Actualmente, la información sobre ocupación de camas, tiempos de espera, uso de quirófanos, consumo de medicamentos y citas médicas se encuentra dispersa en múltiples sistemas y hojas de cálculo desconectadas. Esta fragmentación impide una visión integrada de la operación del hospital, dificultando la toma de decisiones oportunas para optimizar la capacidad instalada, reducir tiempos de espera y garantizar la disponibilidad de medicamentos críticos, impactando directamente en la calidad de atención al paciente y la eficiencia del gasto en salud.*

Antecedentes:

- *El hospital ha crecido en infraestructura y personal médico, pero sus sistemas de información no han evolucionado al mismo ritmo.*
- *Los directivos y jefes de servicio dedican horas a solicitar reportes a TI o a analizar manualmente hojas de cálculo, lo que retrasa decisiones críticas.*
- *Existe una necesidad urgente de contar con datos en tiempo real para gestionar la capacidad hospitalaria, especialmente en épocas de alta demanda.*
- *La información valiosa está "atrapada" en diferentes bases de datos (Historia Clínica, Farmacia, Admisiones), sin una forma ágil de consultarla.*

Alcance del reto: *Los equipos deberán.*

1. *Diseñar y construir un prototipo software funcional (MVP) con un alcance realista para 8 horas de trabajo.*
2. *La solución debe responder claramente al reto planteado mediante un asistente conversacional (Agente IA).*

3. *Incorporar visualizaciones gráficas que faciliten la toma de decisiones (Dashboard) con KPIs hospitalarios clave.*
4. *Permitir la carga y análisis de datos históricos de admisiones, camas, quirófanos y farmacia.*
5. *Generar indicadores clave de gestión hospitalaria (KPI) como:*
6. *Ocupación hospitalaria: Promedio diario/mensual de camas ocupadas por servicio (UCI, Medicina Interna, Pediatría, etc.).*
 - *Tiempos de espera: Tiempo promedio de ingreso a consulta/urgencias por triage.*
 - *Eficiencia de recursos: Uso de quirófanos (cirugías realizadas vs. programadas).*
 - *Consumo de medicamentos: Medicamentos e insumos con mayor y menor rotación.*
 - *Demanda de servicios: Servicios con mayor demanda y especialidades más solicitadas.*
 - *Otros KPIs que consideren relevantes para la gestión hospitalaria.*
7. *Sugerir acciones automáticas o recomendaciones, como:*
 - *Alertas tempranas de desabastecimiento de medicamentos.*
 - *Recomendaciones para reasignar personal o abrir camas en servicios con alta ocupación.*
 - *Optimización de programación de cirugías según disponibilidad de quirófanos.*

Enfoque del desarrollo:

Se recomienda aplicar metodologías o marcos ágiles (Scrum o Design Thinking) para organizar el trabajo por fases (Idear → Prototipar → Testear → Presentar).

Entorno de desarrollo y tecnologías recomendadas

Los participantes pueden elegir libremente sus herramientas, pero deben garantizar compatibilidad, documentación y fácil demostración.

Tipo de solución esperada:

El corazón de la solución será un asistente conversacional (Agente de IA) que permita a directivos, jefes de servicio y personal administrativo realizar consultas en lenguaje natural sobre los datos del hospital para agilizar la toma de decisiones.

Recomendaciones:

a. Frontend

Opción Rápida (Recomendada): Streamlit (Python) - Permite crear dashboards con chat integrado en minutos usando solo Python.

Opción Tradicional: HTML5 + CSS3 + JavaScript con Bootstrap o TailwindCSS y Chart.js para gráficos.

Componentes Clave:

Chatbot/Panel de consulta: Área donde el usuario escribe preguntas en lenguaje natural (ej. "¿Cuántas camas de UCI están ocupadas hoy?").

Dashboard: Tarjetas con KPIs (ocupación, tiempos de espera, stock crítico) y gráficos interactivos.

Visualizador de resultados: Tablas y gráficos que muestran la respuesta del agente.

b. Backend

Lenguaje y Framework: Python con FastAPI (por su velocidad y documentación automática) o Flask (más simple).

Agente de IA (NL2SQL):

Opción Principal: OpenAI API (GPT-3.5-turbo) con prompt engineering que incluya el esquema de la BD y ejemplos (few-shot).

Plan B (Contingencia): Diccionario de preguntas-respuestas SQL predefinidas (si la API falla o no hay acceso).

Endpoints Mínimos:

POST /api/query: Recibe pregunta del usuario → agente genera SQL → ejecuta consulta → devuelve resultados en JSON.

GET /api/kpis: Devuelve datos precalculados para el dashboard.

Validación: Sanear las consultas SQL generadas para prevenir inyecciones.

c. Bases de datos

Opción Rápida (Recomendada): SQLite - No requiere instalación de servidor, es ligero y fácil de reiniciar.

Opción Escalable: PostgreSQL (si se tiene tiempo para configurarlo).

Modelo de Datos Mínimo (3 tablas clave):

Admisiones: id_paciente, fecha_ingreso, fecha_salida, servicio, diagnóstico, triage, médico_asignado.

Camas: id_cama, servicio, estado (ocupada/disponible), id_paciente.

Medicamentos: id_medimento, nombre, stock, fecha_vencimiento, consumo_diario_promedio.

Carga de Datos: Script en Python con Pandas para cargar datos de ejemplo (anónimos) desde CSV.

d. Infraestructura y despliegue

Repositorio: GitHub (público o privado) con README.md completo.

Despliegue Rápido:

Frontend (Streamlit): Streamlit Cloud (gratuito y directo desde GitHub).

Backend: Render o Railway (capas gratuitas).

Base de Datos: SQLite (local) o Supabase (PostgreSQL en la nube).

Ejecución Local: Instrucciones claras en README.md para levantar el proyecto con docker-compose o manualmente.

3. Seguridad

Confidencialidad: Los datos de pacientes deben ser anónimos en la demo. En las respuestas, el agente nunca debe devolver información personal identificable (nombre, documento, diagnóstico específico).

Variables de Entorno: Usar archivo .env para todas las credenciales (API keys, contraseñas). Nunca subir este archivo al repositorio.

Validación: Sanear entradas y consultas SQL generadas para prevenir inyecciones.

Autenticación (Opcional): Si hay tiempo, implementar login con JWT y roles básicos.

e. Colaboración y control de versiones

Gestión del Proyecto: Usar Trello o GitHub Projects con un tablero simple (To Do, Doing, Done).

Control de Versiones: Git con flujo básico (main, feature branches). Hacer commits frecuentes.

README.md: Debe incluir:

Descripción del reto: El problema de gestión hospitalaria.

Instrucciones para ejecutar el proyecto: Paso a paso desde cero.

Tecnologías utilizadas: Lista completa.

Roles del equipo y autores: Quién hizo qué.

f. Buenas prácticas de programación

Código claro, comentado y estructurado.

Nombres de variables y funciones significativos (en inglés).

Modularización: Separar la lógica del agente, la conexión a la BD, la API y el frontend.

Uso de patrones simples como MVC o Factory.

6. Reportes visuales para la toma de decisiones

Dashboard Hospitalario con:

KPIs en tarjetas: Ocupación de camas (%), tiempo de espera promedio (mins), medicamentos con stock bajo.

Gráficos: Usar Chart.js o Plotly para mostrar evolución de ocupación (línea), distribución por servicio (pastel), consumo de medicamentos (barras).

Tablas Dinámicas: Listado de pacientes (sin datos sensibles) con diagnóstico, médico y fecha de ingreso.

g. Innovación tecnológica

El Agente IA (NL2SQL) es la innovación central.

Valor añadido:

Alertas Predictivas: El agente puede anticipar picos de demanda (ej. "Se espera aumento del 20% en ingresos por neumonía. Recomendar aumentar stock de antibióticos X.").

- h. *Análisis de Causa Raíz: Identificar por qué aumentan los tiempos de espera (ej. "El incremento se debe a más pacientes triage 3 en turno noche.").*

Presentación técnica del proyecto:

El equipo debe demostrar el funcionamiento real del prototipo (demo funcional o video), y la presentación debe incluir:

1. **Problema detectado:** *El hospital gestiona su operación con información fragmentada, lo que retrasa decisiones críticas y afecta la calidad de atención.*
2. **Solución propuesta:** *Asistente inteligente (Agente IA con NL2SQL) que permite consultas en lenguaje natural y entrega respuestas inmediatas con datos, gráficos y recomendaciones.*
3. **Arquitectura o diagrama de componentes:**
Frontend (Streamlit/HTML) → Backend (FastAPI) → Agente IA (OpenAI/Reglas) → Base de Datos (SQLite/PostgreSQL).
4. **Tecnologías usadas y justificación:**
Ejemplo: Streamlit para frontend rápido, FastAPI para backend robusto, OpenAI API para NL2SQL, SQLite para datos ligeros.
5. **Limitaciones y posibles mejoras:**
 - *Limitación: Dependencia de API externa, datos de prueba limitados.*
 - *Mejora 1: Usar modelo local (SQLCoder) para privacidad.*
 - *Mejora 2: Integrar con Historia Clínica Electrónica en tiempo real.*
 - *Mejora 3: Implementar machine learning para predicciones más precisas.*
6. **Demostración del prototipo (4 preguntas clave):**
 - Pregunta 1: "¿Cuántas camas de UCI están ocupadas hoy?"*
 - Pregunta 2: "¿Cuáles son los medicamentos con menos de 5 días de inventario?"*
 - Pregunta 3: "¿Cuál es el tiempo de espera promedio en urgencias en la última semana?"*
 - Pregunta 4: "¿Qué servicio tiene más pacientes ingresados este mes?"*

Fuente de datos:

Insumos Hackaton